

Airbus A350-900 | Kapselung

Kategorie	Beschreibung
Definition von Kapselung	Kapselung ist das OOP-Prinzip, bei dem die internen Details eines Objekts verborgen werden und nur notwendige Informationen über öffentliche Schnittstellen zugänglich sind. Dies schützt die Integrität des Objekts und verhindert unerwünschte Veränderungen durch externe Akteure.
Anwendungsfall (Airbus A350-900)	Bei der Modellierung eines Airbus A350-900 werden interne Details jedes Systems wie Engine , FuelSystem oder FlightControlSystem verborgen. Die Systeme bieten nur öffentliche Schnittstellen für den Zugriff auf die wesentlichen Funktionen, was das Risiko von fehlerhaften Eingriffen verringert und die Wartbarkeit verbessert.
Ziele der Kapselung	Datenversteckung: Verbergen von internen Zuständen und Implementierungsdetails. Schutz der Objektintegrität: Verhindern von unzulässigen Änderungen durch externe Einflüsse. Modularisierung: Das System wird in klar abgegrenzte, selbstgenügsame Teile unterteilt.
Beispiel: Kapselung des Triebwerks	Das Engine -System könnte interne Parameter wie fuelConsumption , thrustLevel oder temperature privat halten. Öffentliche Methoden wie <code>increaseThrust()</code> und <code>decreaseThrust()</code> könnten verwendet werden, um diese Parameter zu steuern, ohne direkten Zugriff auf die internen Daten zu erlauben.
Private Attribute	Private Attribute sind Variablen, die innerhalb einer Klasse definiert sind und nur durch die Methoden dieser Klasse zugänglich sind. In einem Engine -System wären interne Parameter wie fuelLevel , operatingTemperature und currentThrust privat und nicht direkt von außen zugänglich.
Öffentliche Schnittstellen	Public Methods ermöglichen es, auf die Funktionalität des Objekts zuzugreifen, ohne die internen Details preiszugeben. Beispielsweise könnte das FlightControlSystem eine öffentliche Methode <code>adjustAltitude()</code> anbieten, ohne dass der Benutzer wissen muss, wie die FlyByWire -Technologie im Hintergrund funktioniert.
Getter und Setter	Getter-Methoden bieten kontrollierten Zugriff auf private Attribute, und Setter-Methoden ermöglichen kontrollierte Änderungen. Zum Beispiel könnte das CabinPressureSystem einen <code>getPressureLevel()</code> -Getter und einen <code>setPressureLevel(int level)</code> -Setter bereitstellen, der nur valide Werte akzeptiert, um fehlerhafte Eingaben zu verhindern.
Beispiel: Kapselung des Treibstoffsystems	Im FuelSystem könnten interne Berechnungen und Parameter wie currentFuelLevel und fuelFlowRate privat gehalten werden. Öffentliche Methoden wie <code>refuel()</code> oder <code>getFuelStatus()</code> könnten verwendet werden, um diese Attribute zu steuern und Informationen zu liefern, während Details wie die Berechnung des Tankvorgangs verborgen bleiben.

Kategorie	Beschreibung
Vorteile der Kapselung	Datenintegrität: Durch die Kontrolle des Zugriffs auf interne Daten wird sichergestellt, dass diese Daten nur in einem gültigen Zustand verändert werden. Fehlerprävention: Der direkte Zugriff auf sensible oder kritische Daten ist verhindert, was zu weniger fehleranfälligem Code führt. Erhöhte Wartbarkeit: Änderungen an der internen Implementierung sind weniger störend für andere Teile des Systems, da nur die öffentlichen Schnittstellen betroffen sind. Modularität: Kapselung ermöglicht eine klare Trennung von Verantwortlichkeiten zwischen den Systemkomponenten.
Vermeidung von direkten Manipulationen	In einem FlightControlSystem wird die Manipulation von internen Parametern wie altitude oder speed direkt durch kontrollierte Methoden durchgeführt, um zu verhindern, dass externe Komponenten diese direkt verändern, was die Systemintegrität gefährden könnte.
Beispiel: Kapselung der Avionik	Die Avionics-Systeme des Airbus A350-900 könnten private, systemkritische Parameter wie navigationData oder sensorReadings intern verwalten. Öffentliche Schnittstellen wie <code>getFlightPlan()</code> oder <code>updateweatherData()</code> könnten es dem Cockpit ermöglichen, sicher mit diesen Informationen zu interagieren, ohne Details zur internen Datenverarbeitung preiszugeben.
Zusammenhang mit anderen OOP-Prinzipien	Abstraktion: Kapselung ergänzt die Abstraktion, da es die detaillierte Implementierung innerhalb eines Objekts verbirgt und nur die relevanten Funktionen zur Verfügung stellt. Vererbung: In vererbten Klassen können öffentliche Methoden die Funktionalität einer Basisklasse erweitern oder überschreiben, aber die interne Logik bleibt durch Kapselung geschützt. Polymorphie: Kapselung sorgt dafür, dass polymorphe Objekte durch eine einheitliche Schnittstelle sicher angesprochen werden können, ohne dass die Implementierungsdetails des Objekts bekannt sein müssen.